



NAME : HAMDAN ALI

WEEK 4 : Day 19

DAY 19

Store and retrieve user data (name, theme preference) using LocalStorage.

Task 19 : Screenshoots Of Output

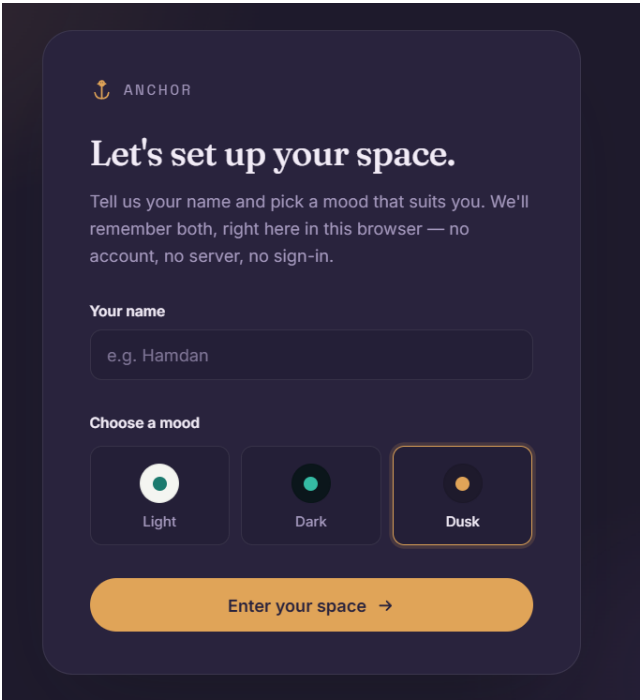
Light Color

The screenshot shows a web form titled "Let's set up your space." with the "ANCHOR" logo. The form includes a text input for "Your name" with the placeholder "e.g. Hamdan". Below it, under "Choose a mood", are three radio buttons: "Light" (selected), "Dark", and "Dusk". At the bottom is a green button labeled "Enter your space →".

Dark Color

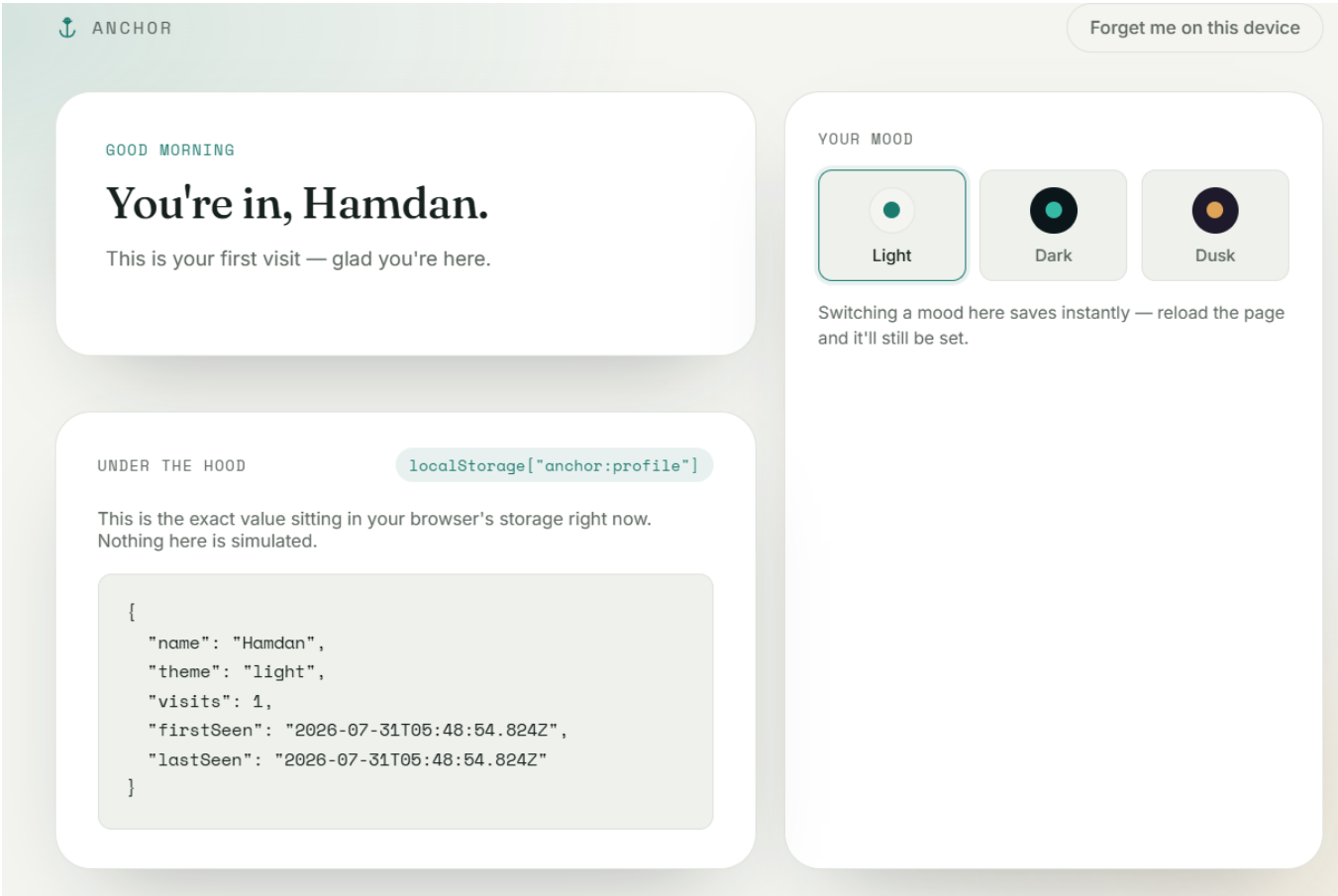
The screenshot shows the same web form as the light theme version, but in a dark theme. The "ANCHOR" logo is teal. The "Let's set up your space." title and instructions are in white. The "Your name" input has a dark background. The "Choose a mood" section has three radio buttons: "Light", "Dark" (selected), and "Dusk". The "Enter your space →" button is teal.

Dusk Color



After Filling The Information:

Result:



ANCHOR

Forget me on this device

GOOD MORNING

You're in, Hamdan.

This is your first visit — glad you're here.

UNDER THE HOOD

localStorage["anchor:profile"]

This is the exact value sitting in your browser's storage right now. Nothing here is simulated.

```

{
  "name": "Hamdan",
  "theme": "light",
  "visits": 1,
  "firstSeen": "2026-07-31T05:48:54.824Z",
  "lastSeen": "2026-07-31T05:48:54.824Z"
}

```

YOUR MOOD

Light

Dark

Dusk

Switching a mood here saves instantly — reload the page and it'll still be set.

EXPLANATION

DAY 19 — LOCALSTORAGE

Anchor — Remembering Someone, One Browser at a Time

Hamdan Ali · Week 4 · Full Stack Web Development Internship

What I Was Going For

The task was to store a name and a theme preference with `localStorage` and read them back. That's genuinely a two-line exercise if you want it to be — one `setItem` call, one `getItem` call. What I wanted to actually understand, though, is why `localStorage` is the tool people reach for in the first place, which only really shows up once there's something worth remembering across visits. So I built a small onboarding flow called Anchor: you tell it your name once, pick a mood out of three themes, and every time you come back — including a hard refresh — it already knows who you are, already looks the way you left it, and can tell you it's your fourth visit and when you were last here.

None of that needs a server. It's all sitting in the browser's own storage, and I wanted the interface to be honest about that instead of hiding it — which is why there's a live panel on the dashboard that shows the exact JSON currently saved, updating the moment anything changes.

One Key, Not Five

`localStorage` only stores strings, and it's tempting to reach for a separate key per value — one for the name, one for the theme, one for the visit count. I went the other way and stored a single key, `anchor:profile`, holding one JSON object with everything in it. Reading and writing both funnel through two small helpers so the `JSON.stringify` and `JSON.parse` only happen in one place each:

```
function readProfile() {  
  const raw = localStorage.getItem(STORAGE_KEY);  
  if (!raw) return null;
```

```
try {
  return JSON.parse(raw);
} catch (err) {
  localStorage.removeItem(STORAGE_KEY);
  return null;
}
function writeProfile(profile) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(profile));
  renderStorageInspector(profile);
}
```

The try/catch in readProfile matters more than it looks. localStorage is just sitting there in the browser, editable by hand through devtools, and if someone breaks the JSON, JSON.parse throws — and if that happens inside an inline script running before the page has even painted, the whole page can fail to load. Treating a broken value the same as no value at all means the worst case is someone gets sent back to onboarding, not a blank white screen.

Solving the Flash of the Wrong Theme

This was the detail I was proudest of figuring out. If you read localStorage and apply the saved theme inside script.js the normal way, that script only runs after the HTML has already been parsed and the page has already painted once — so a returning dark-mode user sees a flash of the light theme for a fraction of a second on every single reload, before it flips to dark. It's a well-known problem, and the fix is to read localStorage earlier than that, in a tiny inline script sitting in the <head>, before the stylesheet even loads:

```
(function () {
  try {
    var raw = localStorage.getItem('anchor:profile');
    var profile = raw ? JSON.parse(raw) : null;
    var theme = (profile && profile.theme) || 'light';
    document.documentElement.setAttribute('data-theme', theme);
    document.documentElement.setAttribute(
      'data-screen',
```

```
    profile && profile.name ? 'dashboard' : 'onboarding'
  );
} catch (err) {
  document.documentElement.setAttribute('data-theme', 'light');
  document.documentElement.setAttribute('data-screen', 'onboarding');
}
})();
```

That script sets two data attributes on the `<html>` element itself — `data-theme` and `data-screen` — and every color in `style.css` is written against `[data-theme="dark"]` and its siblings, so by the time the browser paints anything, it's already the right color and already the right screen. I applied the same idea to which screen shows first: instead of always showing onboarding and then swapping to the dashboard once `script.js` confirms a profile exists, the head script decides that up front too, so a returning visitor never sees the onboarding form flash by.

Letting a Theme Be Previewed Before It's Saved

On the onboarding screen, clicking a theme swatch changes the whole page's colors immediately, before the form is ever submitted — it's a live preview, not a saved choice yet. Only submitting the form actually writes it to `localStorage`. I liked this because it turns the theme picker into something you can genuinely decide with, rather than three labelled circles you're guessing between:

```
onboardForm.querySelectorAll('input[name="theme"]').forEach((radio) => {
  radio.addEventListener('change', () => {
    root.setAttribute('data-theme', radio.value);
    syncActiveThemeOption(onboardForm, radio.value);
  });
});
```

On the dashboard, the same three swatches reappear as plain buttons instead of radio inputs, and clicking one skips the preview step entirely — it previews and saves in the same action, since at that point there's no form to submit, just a preference to update.

Turning a Visit Count Into Something Worth Reading

The task only asked for name and theme, but once I had a JSON object to work with, it felt like a missed opportunity not to also track how many times someone had shown up, and when they were last here — both of which are just more fields in the same object, and both of which make the "we remembered you" feeling much more concrete than a name and a color alone would. Every time the page loads with an existing profile, the visit count increments by one and the previous lastSeen gets kept around just long enough to describe it in words:

```
const previousLastSeen = existingProfile.lastSeen;
const updatedProfile = {
  ...existingProfile,
  visits: (existingProfile.visits || 1) + 1,
  lastSeen: new Date().toISOString(),
  previousLastSeen,
};
writeProfile(updatedProfile);
```

That previousLastSeen value is what lets the dashboard say "you were last here 2 days ago" instead of just "just now" on every load — without keeping the old timestamp around for a moment, there'd be nothing left to compare against once lastSeen gets overwritten.

Making the Storage Itself Visible

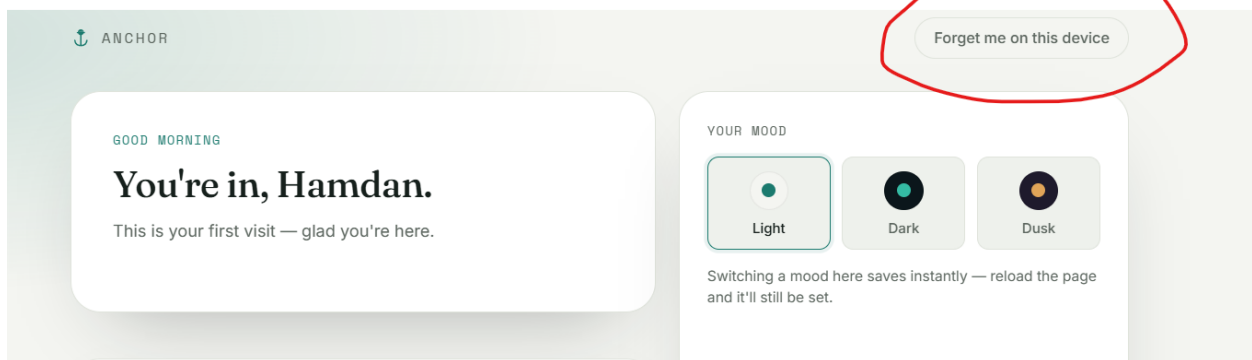
The "under the hood" panel on the dashboard isn't decoration — it's a `<pre><code>` block that gets set to `JSON.stringify(profile, null, 2)` every single time the profile changes, so what you see on screen is always exactly what's sitting in `localStorage['anchor:profile']` at that moment, formatted the same way you'd see it in devtools. I added it mostly for myself while building this, to make sure a theme switch or a visit-count update had actually persisted, but I left it in the final version because it does the same job the W3Schools to-do list article does — it makes `localStorage` feel like a real, inspectable thing instead of a black box you trust and hope works.

Forgetting a Device, on Purpose

Every localStorage exercise focuses on `setItem` and `getItem` and quietly skips `removeItem`, so I made sure Anchor had a real use for it: a "Forget me on this device" button that clears the saved profile entirely and drops the visitor back at onboarding, as if they'd never been here:

```
forgetBtn.addEventListener('click', () => {  
  localStorage.removeItem(STORAGE_KEY);  
  root.setAttribute('data-screen', 'onboarding');  
  root.setAttribute('data-theme', 'light');  
  onboardForm.reset();  
});
```

It's a small thing, but it was worth including — a page that can save your data but never let you remove it doesn't actually demonstrate that you understand the storage, just that you know how to write to it.



A Few Notes on What I Took From This

- localStorage only ever holds strings, so working with anything beyond a single value means `JSON.stringify` going in and `JSON.parse` coming back out — and that parse step can fail, so it needs a try/catch, not an assumption that the data is always well-formed.
- Reading saved preferences in a blocking script before the stylesheet loads is the actual fix for the flash-of-wrong-theme problem — doing it inside the main script file is too late, because the page has already painted once by then.
- One combined key was easier to reason about than several related keys — there was never a moment where the theme and the visit count could disagree with each other, because they were always read and written together.



- Showing the raw stored value on screen, instead of just trusting the save happened, made testing this far faster — every click was something I could immediately verify rather than something I had to take on faith.

References Used

The W3Schools to-do list walkthrough was useful less for the to-do list itself and more for the underlying localStorage pattern it teaches — saving a JSON-serialisable value and reading it back out on load, which is exactly the mechanism behind Anchor's single profile object. The video gave a clearer picture of `setItem`, `getItem`, and `removeItem` working together in a live example rather than in isolated snippets.

Article: [w3schools.com/howto/howto_js_todolist.asp](https://www.w3schools.com/howto/howto_js_todolist.asp)

Video: youtube.com/watch?v=ExDsHljnCyE

Files Submitted

- `index.html` — the onboarding form, the dashboard, and the head script that reads localStorage before first paint.
- `style.css` — the three-mood design system (Light, Dark, Dusk) driven entirely by CSS custom properties and a data-theme attribute.
- `script.js` — reading and writing the profile, live theme previewing, visit tracking, the storage inspector, and the forget-me flow described above.

THANK YOU!